kaneez fatima

Hallucination_Detection_in_Pretrained_Generative_AI_Mode...

hgh
maria
Graphic Era Hill University

Document Details

Submission ID
trn:oid:::1:3609088746

Submission Date
Jul 10, 2026, 5:24 PM GMT+5:30

Download Date
Jul 10, 2026, 5:34 PM GMT+5:30

File Name
Hallucination_Detection_in_Pretrained_Generative_AI_Models_for_Bengali_Text.pdf

File Size
1.7 MB

51 Pages

13,654 Words

76,828 Characters

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Hallucination Detection in Pretrained Generative AI Models for Bengali Text

By

Robiul Islam Tamim (ID: 011212039)
Israt Jahan Rani (ID: 011201376)
Jahinur Islam Tonim (ID: 011212152)
Alif Bin Tanam (ID: 011212098)
Rohit Gupta (ID: 011212148)
Mymuna Nourin (ID: 011212052)

Supervised by

Dr. Ohidujjaman

Associate Professor, Department of Computer Science and Engineering
United International University

Submitted in partial fulfilment of the requirements
of the degree of Bachelor of Science in Computer Science and Engineering

July 9, 2026

Department of Computer Science and Engineering
United International University

Abstract

At present, with the rapid use of Large Language Models (LLMs), the tendency of these models to create hallucinations (the production of the factually incorrect or fabricated in- formation) is increasing. While much research has been carried out for high-resource languages such as English, there is still a huge gap when it comes to dedicated studies and publicly available datasets for low-resource languages (Bangla in particular). Our project seeks to fill this void by creating a sound framework for halluci-nation detection using Bangla LLM outputs. A manually annotated dataset was created containing Bangla prompts, model-generated responses and corresponding hallucination types.For evaluation and fine-tuning we used two multilingual transformer models - **mBERT** and **BanglaBERT**–are utilized. The experiment shows that the **BanglaBERT** outperformed the multilingual baseline with a validation **accuracy** of 82.47% and a macro **F1-score** of 82.45%. **mBERT**, which obtained a macro f1 score of 77.94% and an accuracy of 78.21%. The performance advantage of **BanglaBERT** goes to its domain specific pre-training over Bangla corpora and its ELECTRA based discriminator architecture, which is structurally well suited to a task in which hallucinations are being detected. The end results include a publicly available annotated dataset and fine-tuned model checkpoints, which provide a base resource to enhance the ***credibility and reliability*** of Bengali based AI systems and advance hallucination research in low-resource language settings.

Acknowledgements

This work would have not been possible without the input and support of many people over the course of the project. We would like to express our gratitude to everyone who contributed to it in some way or another.

First, we would like to express our deepest gratitude to our project supervisor, **Dr. Ohidujjaman**, **Associate Professor, Department of Computer Science and Engineering**, **United International University**, for his expert guidance, invaluable insights, and unwavering support throughout every phase of this research. His constructive feedback and encouragement were instrumental in shaping the direction and quality of this work.

We would also like to extend our heartfelt thanks to our course teacher, **Dr. Jannatun Noor Mukta**, **Associate Professor, Dept. of CSE and Director, Data Science Program, United International University**, for her thoughtful instruction, continuous motivation, and the academic foundation she provided, which greatly contributed to the successful completion of this project.

We further extend our gratitude to the Department of Computer Science and Engineering at United International University for providing the computational resources and academic environment that made this research possible.

Above all, we are grateful to **Allah**, the Most Gracious and the Most Merciful, for granting us the health, patience, and strength to complete this project.

We also want to thank our **families** from the bottom of our hearts. Their unconditional love and endless support kept us going through the difficult moments of this work. We hope this achievement brings them as much happiness as it has given us.

Table of Contents

List of Figures

List of Tables

Chapter 1

Introduction

This chapter introduces the project and explains its background. It describes the problem being addressed, the motivation behind the work, the main objectives, the general methodology followed, the expected outcomes, and how the rest of the report is organized.

1.1 Project Overview

The rapid Innovation and mass integration of Large Language Models (LLMs) have transformed how individuals and organizations process and generate information [1]. While these models provide remarkable capabilities in areas such as content creation, translation, and summarization, a critical and Constant struggle remains-the phenomenon of *hallucination* [2]. Hallucination refers to instances where models produce factually incorrect, inconsistent, or fabricated information, which can severely undermine the reliability and trustworthiness of AI-generated outputs [3].

This project seeks to address this issue by developing a robust and reliable framework for hallucination detection, specifically within the context of Bangla LLM outputs . The proposed system employs a fine-tuned **multi-task learning** approach to simultaneously classify the factual correctness of generated responses and identify the specific type of hallucination present. Through this approach, we aim to create a comprehensive solution that not only detects hallucinations but also enhances the overall integrity and practical applicability of Bangla language models [4] .

Prior research has explored hallucination detection through several techniques, including **sampling-based approaches**, **consistency-checking methods**, and **uncertainty-based detection using model ensembles** [5], [6]. However, these efforts have primarily focused on high-resource languages such as English. To the best of our knowledge, no prior studies have specifically addressed hallucination detection in **Bangla**, underscoring the novelty and significance of this work. Figure 3.3 shows a concrete example of hallucination, where an LLM produces a factually incorrect response to a Bangla question.

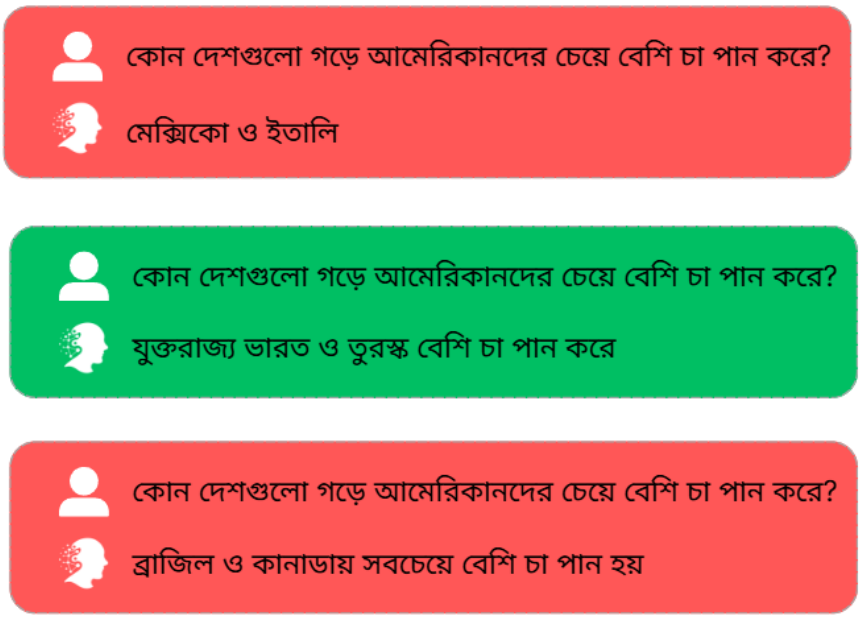

Figure 1.1: The same Bangla question receiving different answers: two hallucinated (red) and one correct (green).

1.2 Motivation

Large language models are applied in numerous areas of our everyday life, such as e-commerce tools, healthcare assistants, digital news media, and education platforms, as well as customer support systems. Although these models do exhibit a good language capability and an ability to effectively understand the context, they have a key limitation that they are prone to - this is called hallucination. This is where a language model will generate the information, which may sound correct and convincing, but is actually incorrect or fabricated.

Significant research has been done to detect and minimize hallucinations in high-resource languages, such as English [7] . However, Bangla is one of the most widely spoken languages in the world with over 230 million speakers, but it is still relatively unexplored in this field of research.

Currently, there are no publicly available benchmarks, annotated datasets or detection frameworks that reliably help to detect Bangla. As a result, millions of users are still exposed to information that is AI-generated and may be unverified or potentially unreliable.

This project is motivated by the need to fill this important research gap by building a systematic benchmark and detection framework to identify hallucinations in Bangla LLM outputs.tion framework for hallucination in Bangla LLM outputs.

1.3 Objectives

The specific objectives of this project are as follows:

1. To build an annotated dataset of the LLM outputs in Bangla, which is manually annotated based on the correctness of the facts and types of hallucination.
2. To fine-tuning two transformer based models, mBERT and BanglaBERT by using the constructed dataset for binary classification of hallucination.
3. To test and compare the performance of the models using the standard evaluation metrics like Accuracy, Precision, Recall, and F1-score.
4. To make the dataset and the fine-tuned models publicly available to advance Bangla natural language processing and detection of hallucinations in the future.

1.4 Methodology

The proposed framework is a seven-stage process as illustrated in Fig. 1.2. Each stage is structured in a way that facilitates ease of repetition and adaptation for other low resource languages/models.

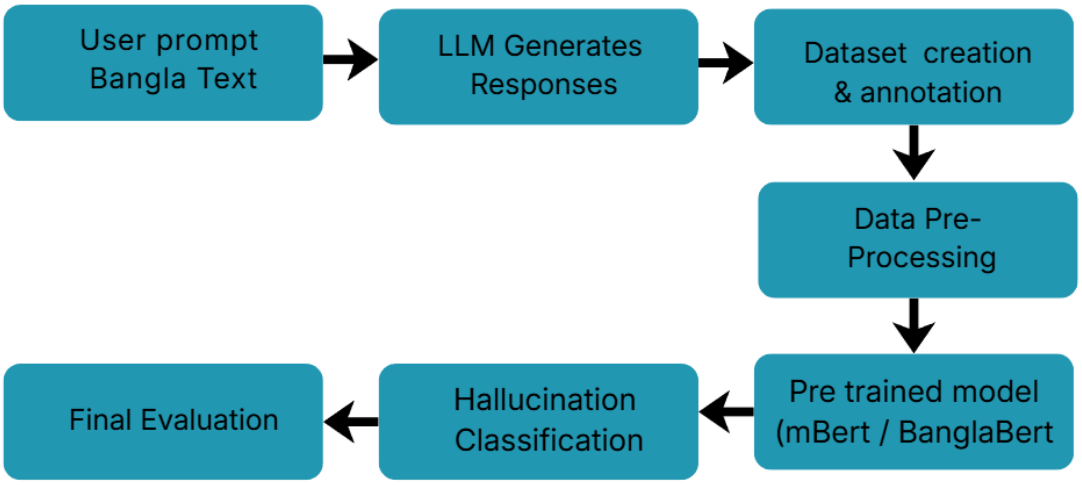

Figure 1.2: shows an overview of the methodology of Bangla hallucination detection.

1. **User Prompt in Bangla Text.** The process starts with Bangla writing in factual questions. The questions are from various areas like Science, History, Current Affairs, Education, Public Issues etc. These are retrieved from credible educational websites, news archives, and open questions to ensure variety of topic and balancing of language usage.

2. **LLM Response Generation.** Each question is submitted separately to three large language models: GPT, Gemini and Claude. Questions are asked one response from each model is recorded. This enables comparing the writing styles and hallucination patterns between systems.

3. **Dataset Creation and Annotation.** All responses generated are manually reviewed by annotators. Each response is checked against known materials of Bangla and bilingual language results to verify the facts and label the type of hallucination when it is present. Responses are assigned a binary label-*Correct* (1) or *Hallucinated* (0)-and a hallucination types where applicable. Annotation consensus protocols are applied for borderline cases. The final dataset comprises **7,043 labelled samples** with six attributes: Bangla question, verified reference answer, LLM-generated response, source model name, binary label, and hallucination type.

4. **Data Pre-Processing.** The annotated dataset undergoes a structured cleaning and preparation phase. This includes deduplication to remove redundant entries, noise removal to eliminate malformed or non-linguistic content, correction of mislabelled samples, and text standardisation. Cleaned samples are then tokenised using model-specific Byte-Pair Encoding (BPE) tokenizers with `[CLS]` / `[SEP]` tokens. The dataset is divided using a stratified 80:20 split, yielding a training set of 5,634 samples and a validation set of 1,409 samples.

5. **Pre-Trained Model Selection.** Two transformer-based models are selected for comparative evaluation: **BanglaBERT** (`csebuetnlp/banglabert`), a language-specific model pre-trained on large Bangla corpora using an ELECTRA-based discriminator objective, and **mBERT** (`bert-base-multilingual-cased`), a massively multilingual baseline pre-trained on 104 languages[8]. A classification head comprising a dropout layer and a linear projection to two output logits is appended to each model's `[CLS]` token representation.

6. **Hallucination Classification.** Both models are fine-tuned on the training set for ten epochs using the AdamW optimizer, a linear learning rate scheduler with warm-up, and a class-weighted cross-entropy loss function to account for the dataset's class imbalance (56.5% Hallucinated / 43.5% Correct).

7. **Final Evaluation.** We use Accuracy, Precision, Recall, and macro F1-Score to evaluate how well the model works on the held-out validation set. To look at how the model works on each label, we make per-class classification reports and confusion matrices.

1.5 Project Outcome

The expected outcomes of this project include:

- A publicly available, manually annotated benchmark dataset of **7,043 Bangla LLM** responses labelled for hallucination.
- Fine-tuned mBERT and BanglaBERT models for Bangla hallucination detection.
- A quantitative performance evaluation showing that BanglaBERT got **82.47%** accuracy and a macro **F1-score** of **82.45%**.
- A framework that may be used again and again as a starting point for more research on how to find hallucinations in low-resource languages.

1.6 Organization of the Report

The remainder of this report is organized as follows. Chapter 2 provides background knowledge on LLMs and hallucination, a review of related literature, and a gap analysis. Chapter 3 shows the project's design, which includes the system architecture, how to make the dataset, and a detailed methodology. Chapter 4 talks about the details of the implementation, the setting for the experiments, and the outcomes of the evaluation. Chapter 5 talks about the design limitations and standards that are important for the project. The paper ends with a summary, limitations, and suggestions for further work in Chapter 6

Chapter 2

Background

This chapter gives the background information needed on Large Language Models and the hallucination phenomenon. It also reviews related research and looks at the gap that this project attempts to address.

2.1 Preliminaries

2.1.1 Large Language Models

LLMs have made natural language processing much better. They are used in many different ways, such as machine translation, automated content generation, intelligent chatbots, and virtual assistants. These models are constructed on transformer-based architectures and trained on huge amounts of text, which lets them give human-like, context-aware answers in a lot of different areas [2, 3]. LLMs are fluent and flexible, but they don't really know what's true and often only rely on learned statistical patterns [6]. Because of this, individuals are likely to create hallucinations, which are writing that sounds accurate but has misleading, made-up, or contextually incongruous information [9]. This problem is quite hard to deal with, especially in languages with few resources like Bangla, where there aren't many dedicated datasets and benchmarks, which makes it extremely harder to find hallucinations and even more important.

2.1.2 Hallucination in LLMs

Hallucination occurs when an LLM makes up information that is untrue, made up, or not related to the situation, but looks and sounds like it is correct to people. When making text, hallucinations might be one of three things:

- **Factual Hallucination:** Information that is completely wrong and goes against facts that can be checked.
- **Partial Hallucination:** A response that has both true and false information.

- **Logical Hallucination:** Sentences that make sense grammatically but don't make sense in the context or have contradictions within them.

Recent studies show that hallucination happens when there are gaps in the training data, the model is too sure of itself, and the generation tactics are not well controlled.This problem makes AI-generated outputs less reliable and less accurate in fields including education, healthcare, and information retrieval [3, 10, 11]. Recent surveys have suggested ways to group different forms of hallucinations and looked into ways to lessen their effects, such as retrieval-augmented generation, confidence calibration, and cross-model verification [5, 12].

2.1.3 mBERT and BanglaBERT

mBERT (Multilingual BERT) is a version of BERT that has been trained on Wikipedia material from 104 languages utilising masked language modelling and next sentence prediction goals [13]. It has great cross-lingual transfer characteristics and works as a general-purpose multilingual model.

BanglaBERT is a BERT model that has been pre-trained on a huge corpus of Bangla text [8].It just looks at Bangla linguistic patterns, hence it does better on downstream Bangla NLP tasks than generic multilingual models. You can get both models from the Hugging Face `Transformers` library.

2.2 Literature Review

2.2.1 Related Research

A significant body of work has explored various methodologies for the detection of hallucinations. Since there is a notable absence of research specifically on hallucination detection in Bangla LLMs, this review will focus on a selection of recent and foundational papers from the English literature to establish a comprehensive understanding of the field.

The theoretical foundation for understanding hallucinations in LLMs has been established by this comprehensive survey, presenting a taxonomy of their types, causes, and challenges [14]. It categorizes hallucinations into three major types that are factuality, faithfulness, and interpretability, as well as some issues open for research in hallucination detection [14]. The same Bangla LLM outputs in this work are based on the above mentioned taxonomy in order to define and organize types of hallucinations.

Another survey goes deep into the issues of hallucinations between generative AI and foundation models with different application areas [15]. t emphasises on reliability in sensitive areas such as healthcare, law and science and discusses some of the mitigation strategies. These domain specific discussions assist in the definition of evaluation criteria and detection strategy considered for Bangla. A different study suggests a statistical

method based on the entropy of semantic representations as sort of a method for detecting hallucinations [16]. t includes the concept of semantic entropy as a means to estimate uncertainty in model outputs and to identify possibly incorrect or inconsistent model responses. Since this method is language independent, it can be used for Bangla text as quantitative indicators of the risk of hallucinations.

Other work includes a comparison of different detection techniques of hallucination in several real world tasks [17]. It tests the consistency-based and semantic-based methods and reports the strengths of these methods under different conditions. These comparative findings are helpful in selecting the suitable techniques for Bangla LLM setting.

An unsupervised framework has been also suggested for detecting hallucinations in real-time by analyzing the activations in the internal model and attention patterns [18]. This approach does not require any labeled data or external references, which is especially helpful for Bangla for which annotated data sets are rare.

Another study is devoted to reduction of these hallucinations on the basis of hierarchical semantic modeling and contrastive decoding [19]. The suggested framework enables improved fact consistency in creating multilingual text.Its ideas can be adapted to Bangla by taking into account language-specific structure and context.

Finally the Hallucination Detectors benchmark introduced in SemEval-2025 introduces a span level hallucination detection approach for English and Arabic [20]. It decomposes model outputs into small syntactic and semantic units and checks the output using inference-based techniques. Although it is effective, this approach requires high-quality reference texts, and correct parsing, which may be difficult with low-resource languages such as Bangla.

2.3 Gap Analysis

Although a number of studies have been done to explore the topic of hallucination detection, there is a clear gap that has still not been bridged. Most available methods, datasets and evaluation frameworks are based on high-resource languages, particularly on English. In comparison, less attention is paid to low-resource languages. For Bangla in particular, no public widespread dataset, nor general purpose framework dedicated to hallucination detection is available.
Bangla has its own grammatical structure, rich vocabulary and a strong cultural context, which need to be investigated in a language-specific manner. To tackle this void, in this work, a publicly available benchmark dataset is introduced and a fine-tuned model is formed specific to the detection of hallucinations in Bangla LLM outputs.

Previous works like Hallucination Detectives [20], ANHALTEN [21] have shown that span-level and token-level detection methods can greatly improve the identification accuracy. These results further encourage design choices followed in this research. Bangla work at the moment does not have any such fine-grained benchmarks, which limits the detection methods' accuracy.

ANHALTEN [21] demonstrates that few-shot and translate-train methods allow cross-lingual adaptation for low-resource languages. Bangla remains unexplored here, and thus the potential of cross-lingual knowledge transfer goes unnoticed.

ASOS [22] and HalluQA [23] highlight that hallucination detection techniques vary across languages, tasks, and model types (span-level vs. claim-level, retrieval-based vs. model-only verification). Bangla work has not yet explored multi-task or multilingual tests, so whether methods trained on other languages will generalize remains uncertain.

These further findings corroborate the need for a specialized, Bangla-oriented hallucination benchmark, robust detection models, and uniform testing procedures, all of which this work aims to provide.

2.4 Summary

This chapter talked about the basic concepts of large language models and hallucination. It reviewed the existing research on the detection of hallucinations in high-resource languages and the lack of research on Bangla. The identified gap essentially influences the purpose of this study and the methodological decisions made in the following chapters.

Chapter 3

Project Design

This chapter describes the design of the Bangla Hallucination Detection Framework. It explains the general system architecture, the dataset generation process, the dataset preprocessing, and the transformer-based classification model that was used to detect hallucinations.

3.1 Requirement Analysis

3.1.1 Functional and Nonfunctional Requirements

Functional Requirements:

- The system shall accept a Bangla question and its corresponding LLM generated response as input.
- The system shall categorize the response into either factual (label: 0) or hallucinated (label: 1).
- The system shall support identification of the type of hallucination: Factual, Partial or Logical.
- The system shall provide the possibility to fine-tune the pre-trained transformer models such as mBERT and BanglaBERT using the curated dataset.
- The system shall output performance metrics including Accuracy, Precision, Recall, and F1-Score.

Nonfunctional Requirements:

- **Accuracy:** The system should achieve at least 70% classification accuracy on the test set.
- **Scalability:** The dataset construction pipeline should be extensible to accommodate more questions and models.

- **Reproducibility:** All experiments must be replicable with the published dataset and model checkpoints.
- **Usability:** The fine-tuned models should be deployable via the Hugging Face API with minimal setup.
- **Linguistic Coverage:** The dataset should include a wide range of topics to make sure it is linguistically rich.

3.1.2 Context Diagram

Figure 3.1 shows where the Bangla Hallucination Detection System sits relative to the people and tools around it. Four external entities interact with the system, each exchanging different kinds of data.

The **Researcher or User** is the primary person driving the process. They feed Bangla question-and-answer pairs into the system and get back a hallucination label along with a confidence score. In practice this covers two roles: someone building and testing the system during development, and someone using the trained model to check AI-generated content later.

The **LLM APIs** (GPT, Gemini, and Claude) sit on the right side of the diagram. The system sends Bangla prompts to them during the data collection phase and receives the raw generated responses that become the dataset. Once the model is trained this connection is no longer needed for inference, but it remains relevant whenever the dataset needs to be extended.

The **Annotation Team** is responsible for labelling. They receive samples from the system, verify them against external factual sources, and return the verified labels. Without their input the supervised fine-tuning cannot happen.

The **Hugging Face API** supplies the pre-trained transformer weights for both BanglaBERT and mBERT. The system downloads those weights at the start of training and, once fine-tuning is complete, pushes the updated checkpoint back for storage and later deployment.

3.1.3 Use Case Diagram

Figure 3.2 maps out what each actor can do with the system. There are four actors: Researcher/User, LLM API, Annotation Team, and Hugging Face API.

The **Researcher/User** has the broadest set of interactions. They submit a Bangla Q&A pair for classification, receive the hallucination prediction back, and are also the actor who triggers the training and evaluation pipeline when building a new model version. The `<<include>>` relationship between *Receive Hallucination Prediction* and *Submit Bangla Q&A Pair* shows that a prediction only ever comes as a result of a submission.

The **Annotation Team** handles the labelling use cases. They assign the binary label (Hallucinated or Correct) and, for everything marked as hallucinated, they also assign the hallucination type (Factual, Partial, or Logical). The `<<include>>` link between these two

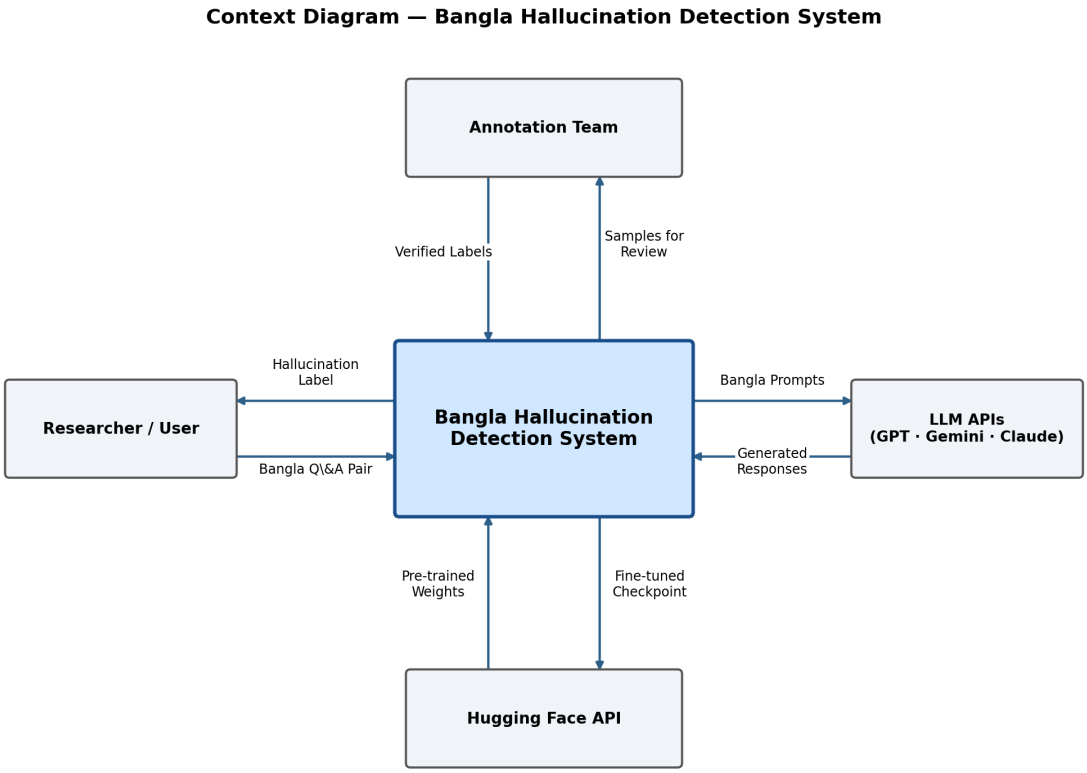

Figure 3.1: Context diagram of the Bangla Hallucination Detection System showing external entities and data flows.

cases captures the dependency: type assignment only applies to rows that have already been labelled as hallucinated.

The **LLM API** actor covers a single use case: generating a response to a given Bangla prompt. This takes place during data collection, before any annotation happens.

The **Hugging Face API** connects to the *Load Pre-trained Model Weights* use case. This is called once at the beginning of each training run, not during inference.

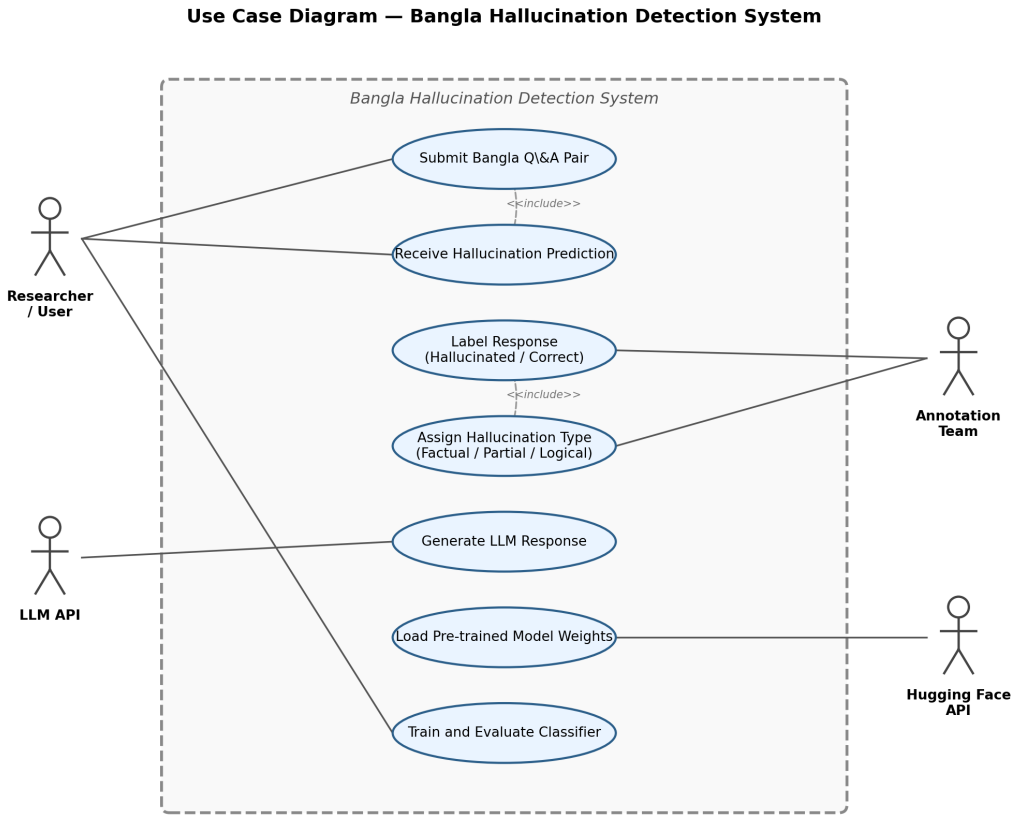

Figure 3.2: Use case diagram showing actors and their interactions with the Bangla Hallucination Detection System.

3.2 Detailed Methodology and Design

In this paragraph, we outline the general approach of the proposed framework as demonstrated in Fig. 3.1. The Bangla Hallucination Detection Framework is designed to detect and categorize on hallucinated reactions provided by LLMs, as well as provide transparency on the model functioning.

Its methodology involves a number of main steps and these steps are data collection, manual annotation, preprocessing, training a transformer-based classifier and performance evaluation. All the stages are important in the process of making sure that the system is able to determine the factual consistency of Bangla LLM outputs. All these measures can be used to establish a more trustworthy and linguistically inclusive AI space.

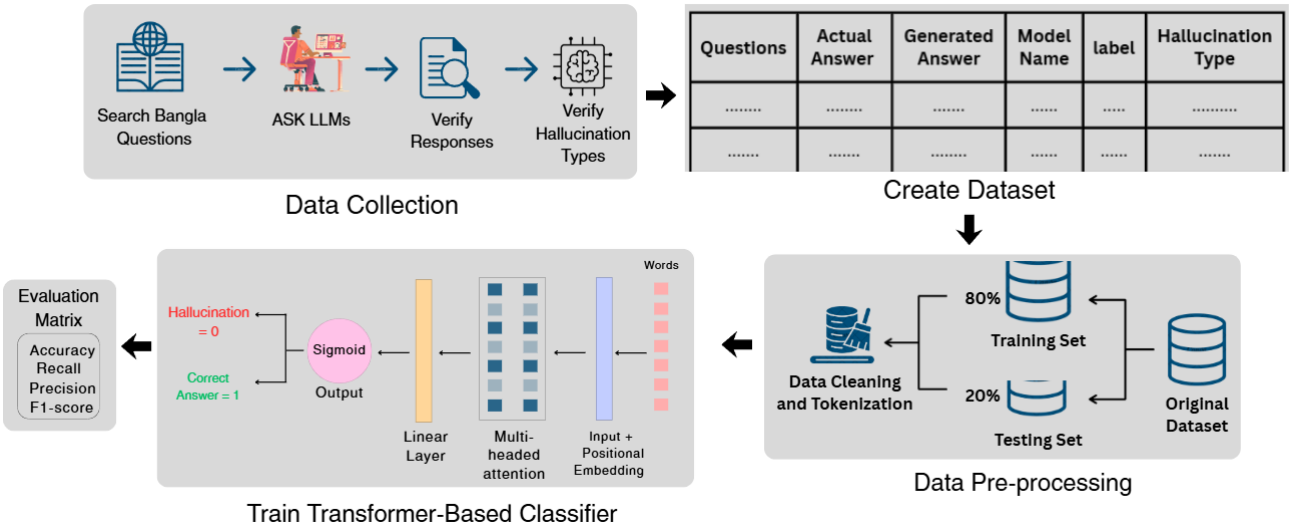

Figure 3.3: Proposed System architecture for Bangla hallucination detection: data collection, dataset creation, preprocessing, and transformer-based classifier training and evaluation.

3.2.1 Dataset Collection and Creation

Question Collection

The questions were from Bangla Wikipedia, online educational sites, general knowledge portals, current affairs pages, and a couple of open quiz repositories that we discovered on the Internet. We went manual testing of each question before incorporating it into the pool The only hard rule was that the question had to be something that had a factual answer, something that could be checked - something that we could confirm or contradict with the use of a reliable source. We also tried to spread the questions over many subject areas rather than pulling everything of one type of source. The seven categories for which we have used are:

- **General Science** - physics, chemistry, biology, and environmental topics
- **History and Geography** - events, dates, places, and notable figures from Bangladeshi and world history
- **Current Affairs** - recent events relevant to the South Asian context
- **Mathematics and Logic** - numerical reasoning and applied problem-solving
- **Culture and Literature** - Bangla authors, festivals, folk traditions, and literary works
- **Health and Medicine** - common medical facts and public health information
- **Technology** - basic computer science, internet usage, topics

We did not want to end up with a situation in which the model could score well by recognising that a certain topic always gets labelled as hallucinated or always gets labelled as correct. Mixing And subjects makes that impossible.

LLM Response Generation

Three models generated the responses: **OpenAI GPT**, **Google Gemini**, and **Anthropic Claude**. Each question was sent to all three in Bangla, with no extra instructions or context added. Just the question, nothing else.

The three responses to each question were saved as three separate rows, each tagged with the model name. That way the dataset carries responses from all three models in roughly equal proportions.

Annotation Process

For each answer, we searched the correct answer using Bangla encyclopaedias, academic sources, and news archives, then read the LLM output against that reference to decide if it was right or wrong.

- **Label (binary):** `0` for hallucinated responses (factually incorrect or fabricated) and `1` for correct responses (factually accurate and verifiable).
- **Hallucination Type:** For responses labelled 0, the specific category of hallucination was recorded:
  - *Factual Hallucination* - the model states something that is simply wrong (wrong date, wrong name, wrong place).
  - *Partial Hallucination* - part of the response is correct but it contains an em- bad data, bedded factual error where the overall accuracy is distorted
  - *Logical Hallucination* - the claims made by the individual may be true in some isolation but the response makes a faulty or misleading conclusion from them.

Factual hallucinations were generally easy to agree on. Partial and logical ones were harder as they involve some judgement about how much of the response is wrong or whether a conclusion follows from the said When we disagreed, we looked at the sample together however and went with whatever the majority decided

Dataset Schema

Each row in the data set contains 6 fields, as described in Table 3.1.

Label Distribution and Dataset Statistics

The total samples for training (full dataset) is 7043 labelled samples. Table 3.2 shows how they break down by class.

Table 3.1: Dataset field descriptions.

Field	Description
Question	It was the original Bangla factual question used as the LLM prompt.
Actual Answer	The verified human checked correct answer drawn from credible sources.
Generated Answer	The raw response the queried LLM generated in response to a given query.
Model Name	The LLM that generated this specific response (GPT, Gemini, or Claude).
Label	Binary annotation: `0` = Hallucinated, `1` = Correct.
Hallucination Type	Factual(fabrication),Partial(Misleading),or Logical(incorrect).

Table 3.2: Label distribution in the final dataset.

Class	Label	Count	Proportion
Hallucinated	0	3,982	56.5%
Correct	1	3,061	43.5%
Total		**7,043**	**100%**

More responses came back as hallucinated than correct – 3,982 versus 3,061. We did not aim for that split; it is just what the labelling produced. Over half the LLM outputs had at least one factual error in them, which was a bit surprising but also makes the case for building this kind of detector. The gap between the two classes is not large enough to need any resampling; we handled it with class-weighting during training, as described in Chapter 4.

We split the data 80:20 for training and validation. To keep the class proportions the same in both halves we used stratified sampling, which gave us 5,634 training samples (3,185 hallucinated, 2,449 correct) and 1,409 validation samples (797 hallucinated, 612 correct).

3.2.2 Data Pre-processing

Once the dataset is finalized, it goes through a comprehensive pre-processing phase to ensure consistency, quality, and suitability for training the classifier [24]. Proper pre-processing is essential to cut down on noise, deal with any biases, and improve model performance .The key pre-processing steps include data cleaning, tokenization, and dataset splitting [24].

Data Cleaning: The first and the most important step in the pre-processing is data cleaning, which is the process of removing errors and unnecessary data from the raw data set. This procedure is very important to make sure that only high-quality, meaningful samples are used for classifier training. There are four major steps in the cleaning process:

1. Deduplication

- **Action:** Finding and getting rid of samples or prompts that are the same.
- **Purpose:**Prevents data from being duplicated, and prevents trainers from putting too much emphasis on certain examples that may change the weights of the model.

2. Noise Removal

- **Action:** Getting rid of elements that do not formatted correctly or that don't have language.
- **Examples:** prompts that have too many special characters, HTML tags or encoding mistakes .
- **Purpose:** To get rid of unnecessary "noise" that leads the model into confusion and stops it from being able to focus on actual text patterns.

3. Correction of Mislabeling

- **Action:** Reviewing and correcting samples that were assigned incorrect categories or labels.
- **Purpose:** Ensures the *integrity* of the ground truth data, preventing the model from learning incorrect associations between input and desired output, which is crucial for predictive accuracy.

4. Standardization

- **Action:** Enforcing uniform formatting across the entire dataset.
- **Examples:** Converting all text to lowercase and ensuring uniform punctuation and spacing conventions.

These steps collectively improve the dataset's reliability by ensuring no misleading or redundant information negatively affects model learning.

Tokenization: Text is tokenized using a pre-trained Byte-Pair Encoding (BPE)-based tokenizer associated with each model. Subword tokenization handles rare Bangla vocabulary. All sequences are padded or truncated to a fixed maximum length L_{max}. Special tokens `[CLS]` and `[SEP]` are added.

Data Splitting: To make sure that the hallucination detection model is trained and tested properly, the dataset is randomly shuffled and split into two parts: a training set and a testing set, with an 80:20 ratio. This technique reduces overfitting and makes sure that the model works effectively with unseen data. The dataset split is designed to maintain the following properties:

1. **Balanced Class Distribution:** This makes sure that both hallucinated and non-hallucinated samples are fairly represented in the training and testing sets.
2. **Stratified Sampling:** Keeps the same number of each type of hallucination in each subgroup.
3. **Randomization:** Randomly shuffles data instances before splitting to eliminate ordering bias and enhance fairness during model learning.

3.2.3 Transformer-Based Classifier

The preprocessed dataset is then used to train a transformer-based classifier for the detection of hallucinations in Bangla LLM outputs. The classifier is designed to separate between factual and hallucinated response using deep learning techniques based on transformer architectures. Transformers are highly popular in NLP because of their capacity to capture long- range dependencies, so are very useful for analyzing linguistic inconsistencies and factual deviations in model generated text. The classifier contains the following key components:

(1) **Input Token Embeddings:** Each textual input (both question and model- generated response) is first tokenized and converted into numerical representations using a pre-trained tokenizer [24]. Given a sequence of input tokens $x_1, x_2, \ldots, x_n$, each token is mapped to a dense vector representation e_i as follows:

$$e_i = \text{Embedding}(x_i), \quad i = 1, 2, \ldots, n[24] \tag{3.1}$$

(2) **Positional Encodings:** These encodings help the model retain the relative positions of words, ensuring proper interpretation of sentence meaning. The positional encoding PE for each token at position i is defined as:

$$\begin{aligned} PE_{(i,2j)} &= \sin\left(\frac{i}{10000^{2j/d}}\right), \\ PE_{(i,2j+1)} &= \cos\left(\frac{i}{10000^{2j/d}}\right)[24] \end{aligned} \tag{3.2}$$

where d is the embedding dimension and j is the position within the embedding vector.

(3) **Multi-Head Attention Mechanism:** The transformer uses a self-attention Multi-Head mechanism to capture dependencies between words across the input sequence [25] [24]. This helps to ensure that the model is focused on the most relevant parts of the

text in deciding if the facts are accurate [26]. The attention scores are calculated with the scaled dot-product attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V[24] \tag{3.3}$$

where Q, K, and V represent the query, key, and value matrices, and d_k is the dimension of the key vectors.

(4) **Feedforward Network and Layer Normalization:** The outputs of the attention are went through a fully connected feedforward network, followed by layer normalization and dropout regularization to avoid overfitting [27] [28] [24]. The feedforward network is comprised of two linear transformations using ReLU activation function [29]:

$$FFN(x) = \max(0, xW_1 + b_1)W_2 + b_2 \tag{3.4}$$

where W_1, W_2 and b_1, b_2 are learnable parameters [29].

(5) **Fully Connected Layers and Output:** The final representations of the hidden states are passed through dense layer which is follow by sigmoid activation function to classify the model-generated responses as "factual" (True), or "hallucinated" (False) [24]. The sigmoid function gives probability score between 0 and 1, gives the likelihood of a response is hallucinated:

$$\hat{y} = \sigma(Wh + b) = \frac{1}{1 + e^{-(Wh+b)}}[24] \tag{3.5}$$

where h is the final hidden representation, W and b are learnable parameters, and σ denotes the sigmoid activation function [30].

The classifier is fine-tuned with the help of supervised learning strategy, where it minimizes binary cross entropy loss function to improve the prediction accuracy [30] [24]. The loss function is defined as:

$$L = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log \hat{y}_i + (1 - y_i)\log(1 - \hat{y}_i)] \tag{3.6}$$

where N is the total number of training samples, y_i is the true label (1 for hallucinated and 0 for factual), and $\hat{y}_i$ is the predicted probability [24].

The model is optimized using AdamW optimizer with weight decay regularization to prevent overfitting [31] [24]. The learning rate is dynamically changed with a linear decay scheduler to make sure stable convergence during the training process.

3.3 Project Plan

1. **Phase 1 - Literature Review and Dataset Design** : Survey relevant literature; define annotation guidelines and dataset schema.

2. **Phase 2 - Dataset Construction**: Collect Bangla questions, query LLMs, manually annotate responses.
3. **Phase 3 - Data Preprocessing**: Clean, tokenize, and split the dataset.
4. **Phase 4 - Model Training**: Fine-tune mBERT and BanglaBERT using the Hugging Face Trainer API.
5. **Phase 5 - Evaluation and Analysis** : Evaluate models, generate confusion matrices, compare performance.
6. **Phase 6 - Report Writing and Public Release**: Write up, publish dataset and models.

3.4 Task Allocation

Team Member	**Responsibilities**
Robiul Islam Tamim	Dataset collection and annotation, mBERT fine-tuning, evaluation, report writing
Israt Jahan Rani	LLM response generation, BanglaBERT fine-tuning, preprocessing pipeline, report writing
Jahinur Islam Tonim	Dataset creation and development
Alif Bin Tanam	Dataset creation and development
Rohit Gupta	Dataset creation and development
Mymuna Nourin	Dataset creation

3.5 Summary

This chapter presented the system design for the Bangla Hallucination Detection Framework, covering requirements, dataset construction methodology, preprocessing pipeline, transformer classifier architecture, project plan, and task allocation. The following chapter details the implementation and experimental results.

Chapter 4

Implementation and Results

This chapter explains the experimental setting, the training setting, the evaluation methodology, and empirical results of Bangla Hallucination Detection Framework. Both transformer-based models - BanglaBERT and mBERT - were fine-tuned on the considered dataset and evaluated on a held-out validation set.

4.1 Environment Setup

All of the experiments were run on Google Colaboratory with an **NVIDIA Tesla T4 GPU** via PyTorch's automatic mixed-precision (AMP) support.The cloud-based environment provided reproducibility and eliminated hardware specific variance.

Some of the important software dependencies are:

- Python (version 3)
- Hugging Face `Transformers` library (version 4)
- PyTorch (with CUDA support)
- `datasets`, `scikit-learn`, `pandas`, `numpy`
- `matplotlib`, `seaborn` (for visualisation)

4.1.1 Dataset Statistics and Class Distribution

On the whole, LLM frameworks, **7,043** samples were collected from three prominent frameworks - GPT. Gemini, Claude), with the distribution in different labels is detailed below:

- **Class 0 - Hallucinated:** 3,982 samples (56.5%)
- **Class 1 - Correct:** 3,061 samples (43.5%)

The data set was divided (Stratified Sampling) in a ratio 80:20:

- **Training set:** 5,634 samples (Class 0: 3,185 - Class 1: 2,449)
- **Validation set:** 1,409 samples (Class 0: 797 - Class 1: 612)

In order to take into account the slight class imbalance, inverse-frequency class weights were applied during training: w0 = 0.8845 (Hallucinated) w1 = 1.1503 (Correct). This penalises mispredictions of the minority class statements more heavily, skewing the models towards balanced learning.

4.1.2 Model Descriptions

Two pre-trained transformer architectures were selected for fine-tuning:

- **BanglaBERT** (`csebuetnlp/banglabert`): An ELECTRA-based discriminator pre-trained on a large, domain-diverse Bangla corpus.
- **mBERT** (`bert-base-multilingual-cased`): A BERT-style encoder pre-trained on 104 languages using masked language modelling.

4.1.3 Training Configuration

The two models were fine tuned using the Hugging Face Trainer API with the following hyperparameters:

- **Epochs:** 10
- **Batch size:** 16 per device
- **Optimizer:** The AdamW optimizer was employed with weight decay regularization.
- **Learning rate scheduler:** a linear decay strategy with a short warm-up phase at the beginning of training
- **Evaluation strategy:** Model evaluation was done after each training epoch
- **Mixed precision:** FP16 precision was enabled to improve memory efficiency and accelerate training.
- **Class weighting:** Used through weighted cross entropy loss

4.2 Evaluation Metrics

The models were tested on the validation split(1,409 samples) using the following standard parameters for classifying:

1. **Accuracy:** Fraction of correct instances over the total number of samples in the dataset:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} [32]$$

2. **Precision:** Fraction of true hypothesized hallucinated responses/ all instances voted as hallucinated:
$$\text{Precision} = \frac{TP}{TP + FP}$$
3. **Recall (Sensitivity):** Percentage of true hallucinated responses that are correctly identified:
$$\text{Recall} = \frac{TP}{TP + FN}$$
4. **F1-Score:** Harmonic mean of Precision and Recall, balancing both concerns:
$$\text{F1} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} [32]$$

where TP, TN, FP, and FN are true positives, true negatives, false positives, And, false negatives, respectively. Both macro-averaged (equal class weight) and weighted-averaged (proportional to class support) variants are reported as the mild class imbalance.

4.3 Results and Discussion

4.3.1 Epoch-wise Training Dynamics

Table 4.1 and Table 4.2 present the per-epoch training and validation metrics for BanglaBERT and mBERT, respectively. Figure 4.1 visualises training and validation loss trajectories for both models, while Figure 4.2 shows the corresponding validation accuracy and F1-score progressions.

Table 4.1: Epoch-wise training and validation metrics for BanglaBERT.

Epoch	Train Loss	Val Loss	Accuracy	F1	Precision	Recall
1	0.6393	0.6147	0.6913	0.6866	0.6892	0.6913
2	0.5319	0.5071	0.7658	0.7639	0.7652	0.7658
3	0.4102	0.4935	0.7807	0.7780	0.7815	0.7807
4	0.3378	**0.4728**	0.7984	0.7989	0.8001	0.7984
5	0.2551	0.5028	0.8119	0.8108	0.8117	0.8119
6	0.1778	0.5273	0.8190	0.8189	0.8188	0.8190
7	0.1503	0.5379	0.8219	0.8218	0.8218	0.8219
8	0.1248	0.5807	0.8212	0.8206	0.8207	0.8212
9	0.1181	0.5869	0.8233	0.8232	0.8231	0.8233
10	0.0873	0.5933	**0.8247**	**0.8245**	0.8244	0.8247

Early-Phase Learning (Epochs 1–3)

Both models show rapid improvement during the first three epochs, but BanglaBERT is ahead from the very first pass. At Epoch 1, BanglaBERT records an accuracy of 69.13%

Table 4.2: Epoch-wise training and validation metrics for mBERT.

Epoch	Train Loss	Val Loss	Accuracy	F1	Precision	Recall
1	0.6848	0.6703	0.6097	0.6047	0.6046	0.6097
2	0.5907	0.6276	0.6991	0.6799	0.7153	0.6991
3	0.5326	0.5664	0.7296	0.7178	0.7407	0.7296
4	0.4439	**0.5439**	0.7466	0.7431	0.7466	0.7466
5	0.4090	0.5809	0.7672	0.7625	0.7704	0.7672
6	0.3002	0.5810	0.7608	0.7601	0.7599	0.7608
7	0.2853	0.6208	0.7701	0.7684	0.7694	0.7701
8	0.1888	0.7458	**0.7821**	**0.7794**	0.7829	0.7821
9	0.1691	0.7400	0.7693	0.7677	0.7686	0.7693
10	0.1762	0.7518	0.7665	0.7648	0.7657	0.7665

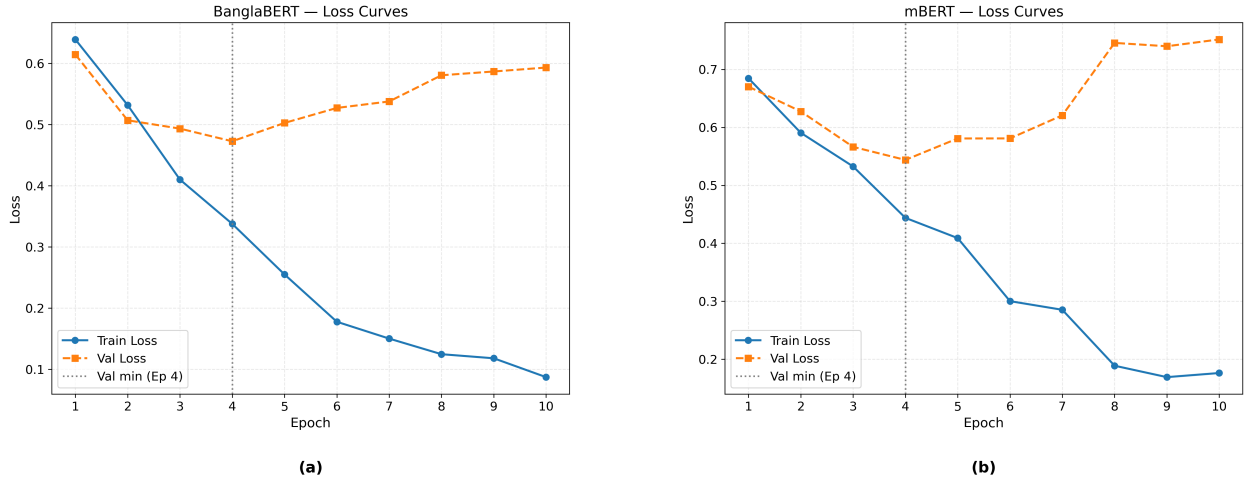

Figure 4.1: Training and validation loss curves over 10 epochs: (a) BanglaBERT, (b) mBERT. Dotted line marks the validation loss minimum.

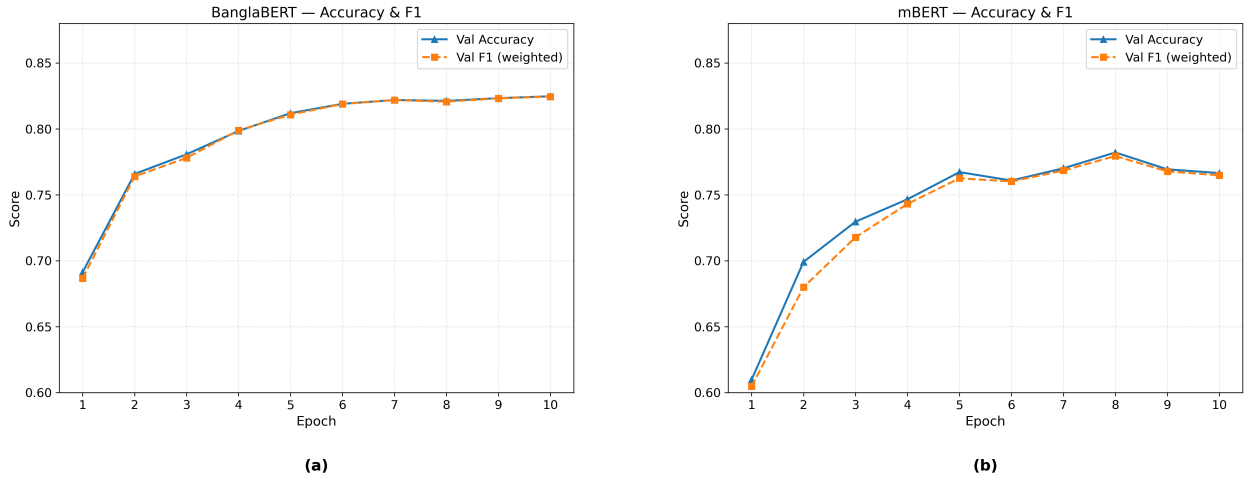

Figure 4.2: Validation accuracy and weighted F1-score per epoch: (a) BanglaBERT, (b) mBERT.

and a validation loss of 0.6147, while mBERT opens at only 60.97% accuracy and a validation loss of 0.6703. That nine-point gap at Epoch 1 is not a quirk of initialisation - it directly reflects BanglaBERT's pre-training advantage. Its weights already carry morphological, syntactic, and semantic patterns drawn from a large Bangla corpus, so from the first gradient update the model is already in a better part of the parameter space for this task. mBERT, whose 110-million parameters are shared across 104 languages, receives much thinner Bangla coverage, and those shared representations must first be adapted before they are truly useful.

By Epoch 2, BanglaBERT's validation loss has dropped sharply to 0.5071 and accuracy has climbed to 76.58%. To put that in perspective, mBERT's best recorded validation loss across its entire Epoch 1 to Epoch 3 window is 0.5664 (at Epoch 3), which still does not match BanglaBERT's Epoch 2 value. On the F1 axis, BanglaBERT's 0.7639 at Epoch 2 is already above mBERT's Epoch 3 F1 of 0.7178. As shown in Figure 4.1, BanglaBERT's validation loss curve descends steeply and smoothly in this early window, while mBERT's curve is noticeably noisier - at Epoch 2, mBERT's validation loss actually *rises* slightly relative to a linearly extrapolated trend, suggesting that the multilingual weight space is harder to navigate efficiently for pure Bangla text.

Precision and recall are practically matched for BanglaBERT throughout this phase (0.7815 precision vs. 0.7807 recall at Epoch 3), which is a sign that the class-weighting strategy is already keeping the model balanced. mBERT, by contrast, shows a 2.1-point precision - recall gap at Epoch 3 (0.7407 precision, 0.7296 recall), reflecting a mild tendency to favour higher precision over capturing all hallucinations.

Mid-Training Convergence (Epochs 4–5)

Both models reach their lowest validation loss at **Epoch 4**: BanglaBERT has a loss of **0.4728** while mBERT has a loss of **0.5439**. The dotted vertical lines in Figure 4.1 mark these points. Interestingly, neither model achieves its highest classification metrics at this exact epoch. BanglaBERT's accuracy at Epoch 4 is 79.84% its F1 score is 0.7989. Both are noticeably lower than its peaks at Epoch 10,which are 82.47% and 0.8245.Similarly, mBERT's accuracy at Epoch 4 is 74.66% , falling short of its peak at Epoch 8 of 78.21%.

It's important to understand the difference between the minimum cross-entropy and the peak classification metrics. Cross-entropy loss penalizes not just incorrect predictions but also overly confident correct ones. As training continues past Epoch 4, both models become more confident in their output probabilities. When a model pushes its softmax outputs closer to 0 or 1, even a correctly classified sample can lead to a higher loss than earlier epochs when the model was less certain. The practical effect is that the decision boundary, which determines classification accuracy and F1 score, can keep improving even as scalar loss increases. Figure 4.2 shows for BanglaBERT: the accuracy and F1 curves rise steadily past Epoch 4 , despite the corresponding increase in validation loss seen in Figure 4.1. The same dynamic holds for mBERT through Epoch 5, where accuracy jumps

to 76.72% even as the validation loss rises to 0.5809.

At Epoch 5, BanglaBERT also crosses the 80% accuracy threshold for the first time, reaching (81.19%).This milestone comes two epochs earlier than mBERT, which does not reach it until Epoch 8. This two-epoch gap clearly shows how much faster BanglaBERT adjusts to the hallucination detection task.

Late-Stage Overfitting (Epochs 6–10)

From Epoch 6 onward, the two models diverge quite sharply in their behaviour. BanglaBERT continues to gain on the validation set. Each epoch brings a small but real improvement in accuracy (81.90% → 82.12% → 82.33% → 82.47%) and F1 (0.8189 → 0.8206 → 0.8232 → 0.8245). The training loss meanwhile falls dramatically - from 0.1778 at Epoch 6 down to 0.0873 at Epoch 10, less than one-seventh of the Epoch 10 validation loss of 0.5933. That gap confirms overfitting: the model has clearly memorised parts of the training distribution. Yet, crucially, whatever patterns are being memorised are transferable enough to the held-out set to keep pushing accuracy upward. This suggests that the training data is diverse enough, and BanglaBERT's pre-trained representations are constrained enough, that additional training continues to extract generalisable signal rather than pure noise.

mBERT's Epochs 6–10 tell a very different story. After peaking at 78.21% accuracy and 0.7794 F1 at Epoch 8, both metrics *regress* - accuracy drops to 76.93% at Epoch 9 and 76.65% at Epoch 10, while F1 falls from 0.7794 to 0.7648. The validation loss, already elevated at 0.7458 by Epoch 8, stays high at 0.7400 and 0.7518 through the final two epochs. This is a clear signal of generalisation breakdown: the model is no longer learning from the data in any way that transfers to unseen samples. All gradient updates from Epoch 9 onward are spent memorising training-set-specific patterns that actually hurt validation performance. Figure 4.2 makes this visible - mBERT's accuracy and F1 curves flatten and then dip after Epoch 8, forming an inverted-U shape that is absent from BanglaBERT's curve.

The final train–validation loss gaps are telling. For BanglaBERT the gap is $0.5933 - 0.0873 = 0.506$; for mBERT it is $0.7518 - 0.1762 = 0.576$, roughly 14% larger. A wider train - validation gap at equivalent training loss levels is a standard indicator of higher overfitting severity, and that is exactly what the numbers show.

Overall Comparison of Training Trajectories

Stepping back and reading Tables 4.1 and 4.2 alongside the curves in Figures 4.1 and 4.2, three structural differences define the contrast between the two models.

The first is the starting gap. At Epoch 1, BanglaBERT is already more than nine percentage points ahead in accuracy. This is not something that training time can fully close, and it frames the entire subsequent trajectory: BanglaBERT is not simply faster, it begins from a stronger initial position.

The second is trajectory shape. BanglaBERT's validation accuracy across ten epochs

forms a smooth, nearly monotonic curve from 69.13% to 82.47%. mBERT's curve rises quickly, levels off, briefly spikes upward at Epoch 8, and then falls back. Non-monotonic validation curves are a practical problem in deployment settings because they make it harder to decide when to stop training and which checkpoint to save; BanglaBERT's behaviour is more predictable.

The third difference is in how well precision and recall stay balanced across epochs. For BanglaBERT, the two values are almost always within one point of each other - for example, 0.7815 precision against 0.7807 recall at Epoch 3. mBERT is less consistent. At Epoch 2 the gap sits at 1.6 points and widens to 2.1 points by Epoch 3 before pulling back after Epoch 4. This matters for hallucination detection because the downstream cost of a false positive (flagging a correct answer) is not the same as the cost of a false negative (missing a hallucination). A model that shifts between the two widely during training is harder to set a reliable threshold for.

Overall, BanglaBERT comes out ahead on every count that matters here: higher numbers, a steadier curve, and a tighter precision–recall spread. Much of that comes down to pre-training on Bangla data specifically, and to the ELECTRA discriminator objective being a better fit for spotting factual inconsistencies than masked language modelling.

4.3.2 Final Model Performance

Table 4.3 reports the final validation metrics for both models on the held-out set of 1,409 samples, and Figure 4.3 puts the numbers side by side visually.

Table 4.3: Final validation performance of BanglaBERT and mBERT on the 1,409-sample held-out set.

Metric	BanglaBERT	mBERT	Difference
Accuracy	**82.47%**	78.21%	+4.26 pp
Precision (Macro)	**82.44%**	78.29%	+4.15 pp
Recall (Macro)	**82.47%**	78.21%	+4.26 pp
F1-Score (Macro)	**82.45%**	77.94%	+4.51 pp
F1-Score (Weighted)	**82.45%**	77.94%	+4.51 pp
Validation Loss	**0.5933**	0.7457	−0.152
Inference Speed	**1358 samp/s**	657 samp/s	≈2×

Looking at Table 4.3, BanglaBERT finishes with 82.47% accuracy against mBERT's 78.21% – a gap of 4.26 percentage points. On its own, a four-point difference might not sound decisive, but when you consider that both models are fine-tuned on the same data with the same class weighting, the gap really does come down to the pre-training: BanglaBERT has seen a large Bangla-only corpus, whereas mBERT spreads its capacity across 104 languages. This difference in representation quality is consistent across all metrics in the table, not just accuracy.

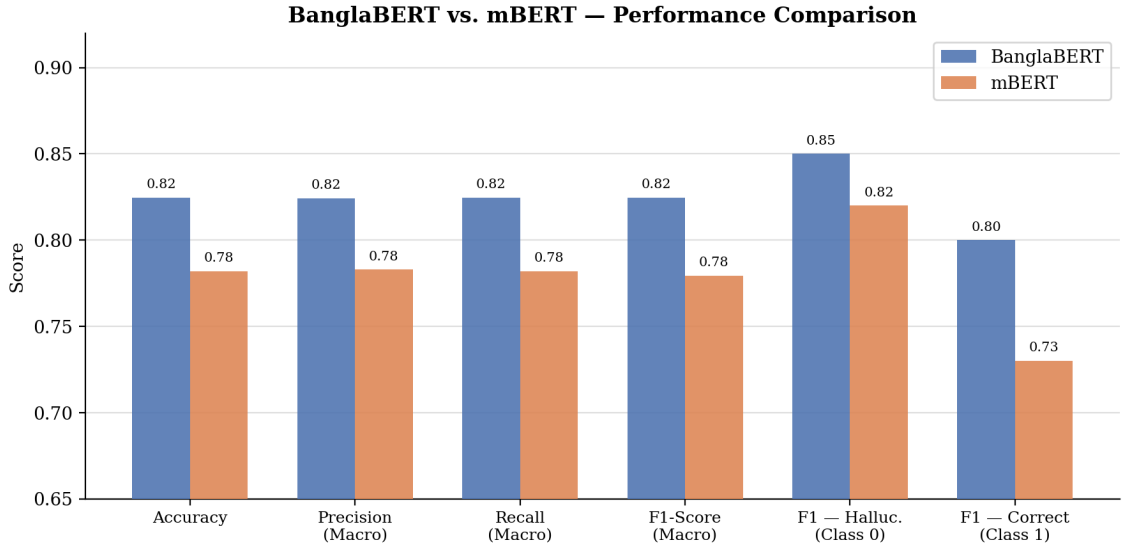

Figure 4.3: BanglaBERT vs. mBERT validation scores across all classification metrics.

The macro F1-score margin of 82.45% compared to 77.94%which is 4.51 percentage points, is the widest in the table and is more significant than accuracy in this case.F1 treats both classes equally, regardless of support. The dataset includes 3982 hallucinated samples versus 3061 correct samples, which results in a – 56/44 split. A model could improve its raw accuracy by simply predicting the majority class. The fact that BanglaBERT's macro and weighted F1 are both at 82.45% shows it is accurately predicting both classes. Figure 4.3 illustrates this spread: all bars for BanglaBERT are above 82%, while for mBERT, the bars are around 78%.

Validation loss is also important to consider. The lower loss for BanglaBERT, at 0.5933 compared to 0.7457, indicates better-calibrated probability estimates. It not only identifies the correct class more often but also does so with higher confidence and is less over-committed when it is uncertain. In a real deployment scenario, better calibration results in more reliable confidence thresholds.

The difference in inference speed, at 1,358 versus 657 samples per second, is notable as well. BanglaBERT is about twice as fast as mBERT on the same hardware. This throughput gap has significant cost implications for a Bangla content moderation or AI quality checking tool in production.

4.3.3 Per-Class Classification Report

Tables 4.4 and 4.5 outline the performance per class for BanglaBERT and mBERT, respectively.

In Table 4.4, BanglaBERT achieves an F1 of 0.85 on the hallucinated class and an F1 of 0.80 on the correct class. Given that there are 3982 hallucinated samples and only 3061 correct ones, a five-point difference between the two class F1 scores is what one would expect and does not suggest any real imbalance in how the model treats the two

Table 4.4: Per-class classification report for BanglaBERT (validation set, $n = 1409$).

Class	Precision	Recall	F1-Score	Support
Hallucinated (0)	0.84	0.85	**0.85**	797
Correct (1)	0.80	0.79	0.80	612
Macro Avg	0.82	0.82	0.82	1409
Weighted Avg	0.82	0.82	0.82	1409

Table 4.5: Per-class classification report for mBERT (validation set, $n = 1409$).

Class	Precision	Recall	F1-Score	Support
Hallucinated (0)	0.78	0.86	0.82	797
Correct (1)	0.79	0.68	**0.73**	612
Macro Avg	0.78	0.77	0.77	1409
Weighted Avg	0.78	0.78	0.78	1409

groups. The precision and recall for the hallucinated class are 0.84 and 0.85, respectively. This means the model captures most actual hallucinations without over-flagging correct answers. The correct class numbers are also comparable: precision is 0.80 and recall is 0.79. No class is sacrificed for another, which is important when the detector reviews actual AI outputs.

Turning to Table 4.5,the recall for the hallucinated class from mBERT is 0.86. At first glance, that number seems good, but the recall for the correct class is only 0.68. In practice, 32 out of every 100 factually correct answers are being flagged as hallucinations. This poses a serious problem for any quality checking use case. About a third of valid content would be incorrectly rejected. The precision figures back this up: mBERT's hallucinated precision is 0.78, while BanglaBERT's is 0.84. mBERT is clearly casting a wider net of predictions on the hallucinated side, resulting in much poorer performance on the correct class. This arises from mBERT's pre-training. Its weights are shared across 104 languages, so Bangla only gets a small portion of its total capacity.

When the model struggles to confidently parse a Bangla sentence, it defaults to predicting the majority class, which on this dataset is hallucinated. In contrast, BanglaBERT was trained solely on Bangla text, so it does not face that limitation. It can understand the sentence rather than merely recognizing surface features, which explains why its correct class recall remains at 0.79 while mBERT's drops to 0.68.

4.3.4 Confusion Matrix Analysis

Figure 4.4 shows the confusion matrices for both models. Each cell gives the raw sample count; the percentages below each count are relative to the full 1,409–sample validation set.

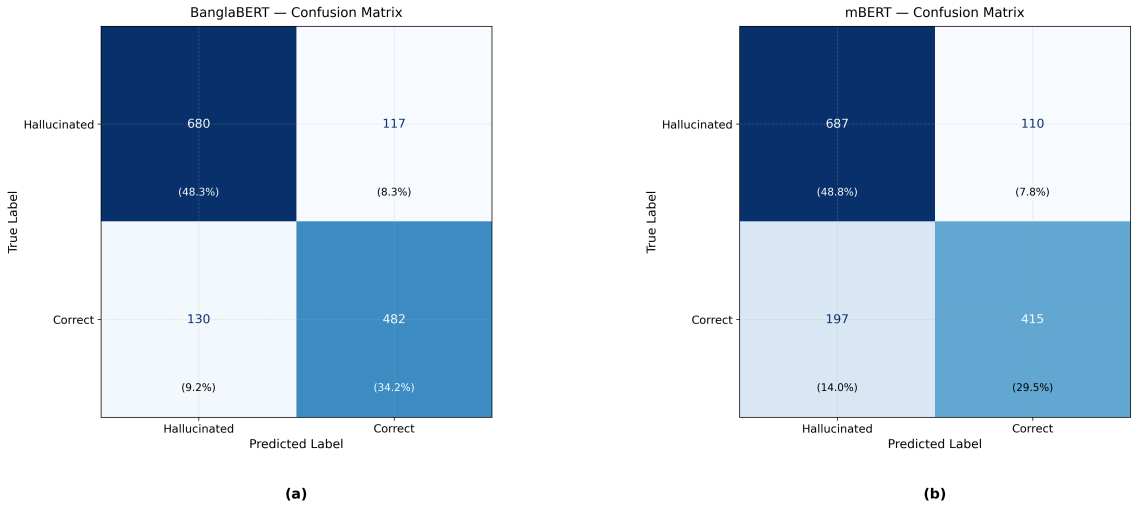

Figure 4.4: Confusion matrices on the 1,409–sample validation set: (a) BanglaBERT, (b) mBERT. Cell percentages are relative to the full validation set.

Starting with BanglaBERT in Figure 4.4(a): of the 797 hallucinated samples, 680 are correctly predicted as hallucinated (85.3%) and 117 slip through as false negatives (14.7%). Of the 612 correct samples, 482 are predicted correctly (78.8%) and 130 are incorrectly flagged as hallucinated (21.2%). So the main error source is false positives – 130 correct answers that the model mislabels. That is 9.2% of the entire validation set. For a binary classifier on a noisy, real–world NLP task, that rate is quite acceptable. The false negative count of 117 (8.3% of the validation set) is similar in magnitude, so neither error type dominates heavily.

The picture in Figure 4.4(b) for mBERT is harder to look at. It catches 687 hallucinated responses correctly (86.2%) – actually slightly more than BanglaBERT's 680 by raw count. But that small gain comes at a steep price: only 415 correct responses are classified correctly (67.8%), while 197 are wrongly flagged as hallucinations (32.2%). That false-positive count of 197 represents 14.0% of the full validation set – more than one and a half times BanglaBERT's false–positive rate of 9.2%. The false negatives tell a slightly better story: only 110 hallucinated samples go undetected, compared to BanglaBERT's 117.

Putting the two matrices side by side in Figure 4.4, the pattern is clear: mBERT trades off Correct–class recall for Hallucinated–class recall. It is aggressive at catching hallucinations but sloppy about what else it catches in the process. BanglaBERT manages to keep both error types at similar, lower rates. From an application standpoint, the false–positive rate is arguably the more critical number. If this model is deployed to review Bangla AI responses, every false positive means a user's correct answer gets rejected or flagged – a direct usability problem. BanglaBERT's 9.2% false-positive rate versus mBERT's 14.0% is a meaningful difference in how disruptive each model would be in production.

4.3.5 Convergence Behaviour and Overfitting Analysis

Both models have a characteristic pattern of initial convergence in validation loss followed by gradual overfitting, but the severity of which and the time of onset is different.

BanglaBERT: Validation loss is minimum at **Epoch 4 (0.4728)** and subse-quently rises to **0.5933** at **Epoch 10** - an increase of 25.4%. Despite this, validation accuracy And F1 keep improving past **Epoch 4** to their highest recorded values at the final epoch. This seemingly contradictory result of increasing validation loss and improving dis- crimination metrics is consistent with models that get more and more confident in their predictions: the cross-entropy loss penalises overconfident softmax predictions even when the binary decision boundary still in good position. The final gap in train-validation loss of 0.506 (training: 0.087, validation: 0.593) confirms the fact that the model has memorised a non-trivial percent of the training distribution.

mBERT: The overfitting pattern of mBERT is more severe. Validation loss is min-imised at **Epoch 4 (0.5439)** and jumps sharply after that, rising to over 0.75 by **Epoch 10** - an increase of 38.4%. Crucially, validation accuracy regresses also after Epoch 8 (78.21 to 76.65%), indicating a total breakdown in the generalisation for the last phase of training .The final gap between train and validation loss for mBERT is 0.575 (training: 0.176, validation: 0.752), having a corresponding gap ∼14% cmore than BanglaBERT, which confirms that mBERT is overfitted had a worse effect on this data set.

The lack of triggered early stopping in both cases means that the stopping criterion was set up against validation loss rather than a classification specific metric, which for BanglaBERT resulted in ongoing training after the point of loss minimisation while dis-crimination of the model in real-world situations continued improving.

4.3.6 Discussion

Both models show fine-tuning pre-trained transformers to be a viable strategy for hallucination detection in Bangla, even if on the dataset scale of approximately 7,000 sam- ples. The results set a quantitative performance benchmark for the community and provide concrete empirical support for the use of BanglaBERT as the main hallucination detector in the proposed framework.

4.4 Summary

This chapter outlined the experimental setup, training configuration and evaluation results for the Bangla Hallucination Detection framework. BanglaBERT achieved superior performance for all measures (82.47% accuracy, 82.45% macro F1-score, and approxi-

mately twice the throughput of inference of mBERT-with a more balanced per-class detection profile.

Chapter 5

Standards, Design Constraints, and Engineering Analysis

This chapter addresses the compliance to relevant software and hardware standards, the design constraints that apply to the Bangla Hallucination Detection Framework, and maps the project to the complex engineering problem-solving and activity categories.

5.1 Compliance with Standards

5.1.1 Software Standards

- **Python PEP 8:** All Python source code follows PEP 8 style guidelines, ensuring readability, consistent indentation, and maintainable codebases.
- **Hugging Face Model Hub Standards:** Both BanglaBERT and mBERT are trained and exported in formats fully compatible with the Hugging Face `Transformers` library, enabling straightforward deployment via the Inference API or a custom-hosted endpoint.
- **Unicode (UTF-8):** All Bangla text-questions, reference answers, and generated responses-is stored in UTF-8 encoding, ensuring consistent character representation across operating systems and platforms.
- **CSV / JSON Data Formats:** The dataset is distributed in standard CSV and JSON formats, ensuring broad accessibility across data science toolchains .
- **Open-Science Licensing:** The dataset and fine-tuned model checkpoints are released under a permissive open-source licence , in keeping with open-science and FAIR data principles.
- **Reproducibility Standard:** Experiments are fully reproducible via fixed random seeds, publicly available dataset splits, and model checkpoints hosted on the Hugging Face Hub.

5.1.2 Hardware and Compute Standards

- **NVIDIA CUDA:** Training leverages NVIDIA CUDA for GPU-accelerated computation on the Tesla T4 GPU. Mixed-precision FP16 training was applied throughout, reducing memory footprint by approximately 50% without measurable accuracy degradation.
- **Google Colaboratory:** Cloud-based training was conducted on Google Colab's free-tier T4 GPU. This choice aligns with the reproducibility standard-any researcher with a Google account can replicate the experiments without local GPU infrastructure.

5.1.3 Communication Standards

- **HTTPS / REST API:** External LLM APIs (OpenAI GPT, Google Gemini, Anthropic Claude) were queried over HTTPS using REST standards, ensuring encrypted and authenticated data transfer.
- **Hugging Face Inference API:** Model inference is compliant with the Hugging Face Hosted Inference API specification, allowing the deployed BanglaBERT model to be accessed via a standard JSON REST interface.

5.2 Design Constraints

5.2.1 Economic Constraint

The project was carried out with limited financial resources. External LLM API calls (GPT, Gemini Pro, Claude) have per token charges, limiting the amount of pro- grammatically produced responses. A combination of free tier API credits and manual need for annotation used to stay within budget. Model weights: Open source (BanglaBERT, mBERT) were used throughout in order to avoid proprietary licensing costs. Total external API expenditure for response generation (estimated to be $15-30 USD).

5.2.2 Environmental Constraint

GPU-based deep learning training has an energy cost. By conducting all training on Google Colab's shared cloud infrastructure (instead of dedicated on-premise hardware is running continuously), the project minimises idle power draw. FP16 training reduces the time for computations per epoch, reducing GPU time; Given 10 training epochs on a T4 GPU with approx. power envelope 70W, total training energy consumed by both models is estimated at less than 0.5 kWh - a negligible carbon contribution compared to standard data centre workloads.

5.2.3 Ethical Constraint

- **Bias Mitigation:** Dataset questions were based on different domains - education, science, history, current events, and culture-to avoid topical skew which can advantage or disadvantage reactions from particular LLMs.
- **Multi-LLM Balance:** The responses were gathered from three major commercial LLMs (GPT, Gemini, Claude) in a rotating fashion (to ensure no single model's output style is predominant in the hallucination patterns in the dataset.
- **Annotation Fairness:** Factual labels were given based on objective verifi- This is a problem with our anti-Bangla language policy where "credible" means "any reference sources other than Bangla and bilingual." Annotation consensus protocols were used to reduce subjective labelling by borderline cases bias.
- **Data Privacy:** No personally identifiable information (PII) was collected, generated or kept in the dataset. All questions are related to general, publicly verifiable factual topics.
- **Misuse Prevention:** The hallucination detector is intended for constructive use- to help improve the factual reliability of Bangla A.I systems. It must not be used for another purpose to facilitate information suppression, censorship or targeted content removal.

5.2.4 Health and Safety Constraint

No direct health or physical safety risks are involved with this software project.

5.2.5 Social Constraint

The Bangla language is spoken by more than 230 million people in Bangladesh as well as West Bengal, And the Bangla diaspora all over the world. The growth of content created by AI in Bangla- without sufficient hallucination detection - has substantial risks to public information quality in a community where digital literacy tools and NLP infrastructure are still com- paratively limited. This project directly responds to that social responsibility to provide an open, community accessible framework for enhancing the factual integrity of Bangla AI outputs.

5.2.6 Sustainability

The dataset, model checkpoints and training code are publicly released in order to maximise research value in the longer term. The framework is modular by design: new questions, additional LLM sources, or revised annotation taxonomies can be incorporated with minimal overhead. This extensibility ensures that the framework remains relevant and actively useful as both the Bangla NLP landscape and the capabilities of large language models continue to evolve.

5.3 Cost Analysis

Item	Cost (USD)	Notes
LLM API usage (response generation - GPT, Gemini, Claude)	$15–30	Estimated based on token volume across 7,043 samples
GPU compute (Google Colab T4)	$0	Free-tier Colab used for all training
Electricity (local pre-processing)	∼$1–3	Minor; data cleaning and notebook development
Annotation labour	$0	Performed by project members
Model hosting (Hugging Face Hub)	$0	Free-tier public model repository
Total Estimated Cost	**$16–33**	

5.4 Complex Engineering Problem

5.4.1 Complex Problem Solving

- **P1 - Depth of Knowledge:** The work requires solid understanding of Bangla linguistics, transformer architectures (ELECTRA-based BanglaBERT vs. multilingual BERT), and FP16 mixed-precision training - none of which are routine undergraduate-uate topics.
- **P2 - Conflicting Requirements:** TThere is a direct tension between maximising remember (catching more hallucinations) and keep false positive low (not flagging correct answers). This trade-off was monitored in terms of confusion matrices for both models.
- **P3 - Depth of Analysis:** Training required per-epoch monitoring of loss curves, overfitting checks, and per-class F1 breakdown - not just final accuracy figures.
- **P7 - Inter-dependence:** Dataset quality, class weighting decisions, and model architecture are very closely coupled. A shift in any one of them causes a shift in the overall F1 score so they had to be tuned together.

Table 5.1 maps these problem attributes.

5.4.2 Engineering Activities

- **A1 - Range of Resources:** The work was based on using a Tesla T4 GPU (via Google Colab), three commercial LLM APIs (GPT, Gemini, Claude), manual human annotation, and and the Hugging Face model and dataset ecosystem.

Table 5.1: Mapping of the project with complex problem-solving criteria.

P1 Depth of Knowledge	P2 Range of Conflicting Requirements	P3 Depth of Analysis	P4 Familiarity of Issues	P5 Extent of Applicable Codes	P6 Stakeholder Involvement	P7 Interdependence
√	√	√	×	×	×	√

- **A2 - Level of Interaction:** Annotators were involved directly in labelling decisions. Labelling guidelines were developed progressively through group discussion to decide on disagreements on borderline cases.
- **A3 - Innovation:** This project produced the first publicly available Bangla hallucination detection dataset and the first head-to-head benchmarking between BanglaBERT and mBERT for this task.
- **A4 - Societal and Environmental Consequences:** Better hallucination detection directly raises the factual reliability of Bangla AI tools used by over 230 million speakers. On the environmental side, using cloud FP16 training kept energy consumption low.

Table 5.2 maps the project's primary activities to the complex engineering activity criteria.

Table 5.2: Mapping of the project with complex engineering activity criteria.

A1 Range of Resources	A2 Level of Interaction	A3 Innovation	A4 Societal & Environmental Consequences	A5 Familiarity
√	√	√	√	×

5.5 Summary

This chapter described the software and hardware standards to be used, identified and contextualised major design constraints, economic and ethical, social and environmental, and political, as well as a detailed cost analysis. The project was mapped to Washington Accord complex engineering problem-solving (P1, P2, P3, P7) and activity (A1 - A4) criteria demonstrating the breadth and depth of engineering rigour applied throughout. The next chapter consists of the conclusions of the project, limitations, and further works.

Chapter 6

Conclusion

This chapter summarises the contributions of the project, discusses limitations encountered, and suggests directions for future work.

6.1 Summary

This project created the first publicly-available framework for hallucination detection in Bangla large language model (LLM) outputs. A manually annotated dataset of **7043 Bangla question - response pairs** were curated from 3 major LLMs - (GPT, Gemini, Claude) cover a wide range of factual domains. Each sample was labelled for binary factual correctness (Hallucinated / Correct) as well as hallucination types. Two pre-trained transformer models - **BanglaBERT** (csebuetnlp/banglabert) and **mBERT** (bert-base-multilingual-cased) were fine-tuned on this data set for ten epochs on a Tesla T4 GPU with mixed precision (FP16) training, and evaluated on a stratified held-out validation set of 1409 samples.

BanglaBERT proved to be the hands-down winner of the model with **82.47%** validation accuracy and a macro F1-score of **82.45%**, and inference throughput of about 1,358 samples computed per second. **mBERT** had an accuracy of 78.21% and macro F1-score of 77.94%, being a competitive multilingual baseline. The performance gap was most pro-scored in the F1-score for the Correct class (BanglaBERT: 0.80 vs. mBERT: 0.73), where mBERT's tendency to over-predict Hallucinated label results in a substantially increased false positive rate (32.2The results confirm that fine-tuning pre-trained, language-specific transformer models is an effective and resource-efficient hallucination detection strategy for low-resource languages. The ELECTRA-based discriminator architecture of BanglaBERT provides a structural advantage to capture factual inconsistencies because its pre-training objective is structurally similar to the task of detecting hallucinations, The published data set and fine- tuned model checkpoints as the basis of benchmark for the Bangla NLP research community.

6.2 Limitation

- **Dataset Size:** Despite the dataset being expanded to 7,043 samples, it is still humble in comparison to English-language markers. Further scaling-particularly for domain-specific content-would presumably increase robustness of models and generalistion.
- **Annotation Subjectivity:** Despite the use of consensus-based annotation, the classification of Partial and Logical hallucinations, inherent subjectivity, which may introduce noise.
- **Overfitting:** We can see that both models have an increasing train-validation loss difference beyond epoch Early stopping was set up against validation loss and not F1-score, which causes carried on training beyond the point of optimal generalisation. Future experiments should use F1-based early stopping with cosine learning rate scheduler in order to mitigate this.
- **Domain Coverage:** The dataset are mostly general facts; domain- specific content (medical, legal) is underrepresented.

6.3 Future Work

There are several directions that can develop and extend this framework:

1. **Dataset Expansion:**Gathering a larger and more diverse dataset of Bangla language, include- ing colloquial expressions, regional dialects and content (medical, legal, scientific), will enhance model generalisation and coverage.
2. **Fine-grained Detection:** Implementing span or token level hallucination de- tection will offer greater resolution insights.
3. **Ensemble and Multi-Task Learning:** Ensemble of mBERT and BanglaBERT outputs using ensemble learning, or using multi-task learning to simultaneously classify binary labels and types of hallucinations, could produce better performance.
4. **Multimodal Hallucination:** Exploring the study of hallucination in cross-modal situations (text in combination with audio/visual data) is more relevant for modern AI applications.
5. **Explainability:** Using attention visualisation and SHAP explainability methods will help users. acquire knowledge of model decisions and build trust.

References

[1] José Valente de Oliveira, João Leite, João Rodrigues, João Dias, and Pedro Cardoso. *Progress in Artificial Intelligence: 24th EPIA Conference on Artificial Intelligence, EPIA 2025, Faro, Portugal, October 1–3, 2025, Proceedings, Part II*. Springer Nature, 2025.

[2] Yuxia Wang, Minghan Wang, Muhammad Arslan Manzoor, Fei Liu, Georgi Georgiev, Rocktim Jyoti Das, and Preslav Nakov. Factuality of large language models: A survey. *arXiv preprint arXiv:2402.02420*, 2024.

[3] Aisha Alansari and Hamzah Luqman. A comprehensive survey of hallucination in large language models: Causes, detection, and mitigation. *arXiv preprint arXiv:2510.06265*, 2025.

[4] Ziwei Ji. *Why do Machines Make up Stuff? Hallucination in LLMs*. PhD thesis, Hong Kong University of Science and Technology (Hong Kong), 2025.

[5] Haoqiang Kang, Terra Blevins, and Luke Zettlemoyer. Comparing hallucination detection metrics for multilingual generation. *arXiv preprint arXiv:2402.10496*, 2024.

[6] Bo Zhou, Daniel Geißler, and Paul Lukowicz. Misinforming llms: vulnerabilities, challenges and opportunities. *arXiv preprint arXiv:2408.01168*, 2024.

[7] Fahad J Abdu, Raed Mughaus, Shadi Abudalfa, Moataz Ahmed, and Ahmed Abdelali. An empirical evaluation of arabic text formality transfer: a comparative study. *Language Resources and Evaluation*, 59(4):4093–4153, 2025.

[8] Mahmud Kabir Yousuf, Mahmudul Hasan Rifat, Pronoy Kumar Mondal, Anmay Paul Arpan, Md Mahmudul Islam, and Md Sadekur Rahman. Addressing class imbalance in bengali sentiment analysis: A comparative study of banglabert and multilingual bert. In *2025 International Conference on Electrical, Computer and Communication Engineering (ECCE)*, pages 1–7. IEEE, 2025.

[9] Wan Zhang and Jing Zhang. Hallucination mitigation for retrieval-augmented large language models: a review. *Mathematics*, 13(5):856, 2025.

[10] SMTI Tonmoy, SM Zaman, Vinija Jain, Anku Rani, Vipula Rawte, Aman Chadha, and Amitava Das. A comprehensive survey of hallucination mitigation techniques in large language models. *arXiv preprint arXiv:2401.01313*, 6, 2024.

[11] Hanzhi Zhang, Sumera Anjum, Heng Fan, Weijian Zheng, Yan Huang, and Yunhe Feng. Poly-fever: A multilingual fact verification benchmark for hallucination detection in large language models. *arXiv preprint arXiv:2503.16541*, 2025.

[12] Xinxin Liu. A survey of hallucination problems based on large language models. *Applied and Computational Engineering*, 97:24–30, 2024.

[13] Mashael Al-Duwais, Hend Al-Khalifa, and Abdulmalik Al-Salman. A benchmark evaluation of multilingual large language models for arabic cross-lingual named-entity recognition. *Electronics*, 13(17):3574, 2024.

[14] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, et al. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems*, 43(2):1–55, 2025.

[15] Pegah Ahadian and Qiang Guan. A survey on hallucination in large language and foundation models. *Preprints*, 2025.

[16] Sebastian Farquhar, Jannik Kossen, Lorenz Kuhn, and Yarin Gal. Detecting hallucinations in large language models using semantic entropy. *Nature*, 630(8017):625–630, 2024.

[17] Gaurang Sriramanan, Siddhant Bharti, Vinu Sankar Sadasivan, Shoumik Saha, Priyatham Kattakinda, and Soheil Feizi. Llm-check: Investigating detection of hallucinations in large language models. *Advances in Neural Information Processing Systems*, 37:34188–34216, 2024.

[18] Weihang Su, Changyue Wang, Qingyao Ai, Yiran Hu, Zhijing Wu, Yujia Zhou, and Yiqun Liu. Unsupervised real-time hallucination detection based on the internal states of large language models. *arXiv preprint arXiv:2403.06448*, 2024.

[19] Yanyi Liu, Qingwen Yang, Jiawei Tang, Tiezheng Guo, Chen Wang, Pan Li, Sai Xu, Xianlin Gao, Zhi Li, Jun Liu, et al. Reducing hallucinations of large language models via hierarchical semantic piece. *Complex & Intelligent Systems*, 11(5):1–19, 2025.

[20] Passant Elchafei and Mervat Abu-Elkheir. Hallucination detectives at semeval-2025 task 3: Span-level hallucination detection for llm-generated answers. In *Proceedings of the 19th International Workshop on Semantic Evaluation (SemEval-2025)*, pages 601–606, 2025.

[21] Janek Herrlein, Chia-Chien Hung, and Goran Glavaš. Anhalten: Cross-lingual transfer for german token-level reference-free hallucination detection. *arXiv preprint arXiv:2407.13702*, 2024.

[22] Serry Taiseer Sibaee, Abdullah I Alharbi, Samar Ahmed, Omar Nacar, Lahouri Ghouti, and Anis Koubaa. Asos at arabic llms hallucinations 2024: Can llms detect their hallucinations. In *Proceedings of the 6th Workshop on Open-Source Arabic Corpora and Processing Tools (OSACT) with Shared Tasks on Arabic LLMs Hallucination and Dialect to MSA Machine Translation@ LREC-COLING 2024*, pages 130–134, 2024.

[23] Qinyuan Cheng, Tianxiang Sun, Wenwei Zhang, Siyin Wang, Xiangyang Liu, Mozhi Zhang, Junliang He, Mianqiu Huang, Zhangyue Yin, Kai Chen, et al. Evaluating hallucinations in chinese large language models. *arXiv preprint arXiv:2310.03368*, 2023.

[24] Md. Faiyaz Abdullah Sayeedi, Maaz Bin Hossain, Md. Kamrul Hassan, Sabrina Afrin, Molla Md. Sabit Hossain, and Md. Shohrab Hossain. Jailbreaktracer: Explainable detection of jailbreaking prompts in llms using synthetic data generation. *IEEE Access*, 13:123708–123723, 2025.

[25] SP Jani and M Adam Khan. *Applications of AI in Smart Technologies and Manufacturing: Volume 2*. CRC Press, 2025.

[26] Pushpa Choudhary, Sambit Satpathy, Arvind Dagur, and Dhirendra Kumar Shukla. *Recent Trends in Intelligent Computing and Communication: Volume 1*. CRC Press, 2025.

[27] Farima Fatahi Bayat. *Evaluating and Enhancing Language Model Factuality*. PhD thesis, 2025.

[28] F Xu, H Uszkoreit, Y Du, W Fan, D Zhao, and J Zhu. Natural language processing and chinese computing. *Lect Notes Comput Sci*, 11839:563–574, 2019.

[29] Antonios Papaleonidas, Elias Pimenidis, Harris Papadopoulos, and Ioannis Chochliouros. *Artificial Intelligence Applications and Innovations. AIAI 2025 IFIP WG 12.5 International Workshops: MHDW 2025, AI4GD 2025, and SilverTech 2025, Limassol, Cyprus, June 26–29, 2025, Proceedings, Part II*. Springer Nature, 2025.

[30] CHINA) INTERNATIONAL CONFERENCE ON INTELLIGENT COMPUTING (21ST: 2025: NINGBO. *ADVANCED INTELLIGENT COMPUTING TECHNOLOGY AND APPLICATIONS*. SPRINGER, 2025.

[31] R Udayakumar and R Manimaran. Experimental and cfd analysis on cutting tool temperature with different cooling techniques. In *Computational Techniques and Smart Manufacturing*, pages 244–253. CRC Press, 2026.

[32] Maryam Tahmooresi, A Afshar, B Bashari Rad, KB Nowshath, and MA Bamiah. Early detection of breast cancer using machine learning techniques. *Journal of Telecommunication, Electronic and Computer Engineering (JTEC)*, 10(3-2):21–27, 2018.